

WIPRO EMBEDDED TRAINING PROGRAM

Individual Capstone Project • 6-Stage SDLC Technical Report

VDEVPULSE

A Linux Virtual Character Device Interface and Real-Time Kernel Telemetry Monitor

Candidate Name: Priyansu Das

Training Track: Embedded Systems & System Software

Language & Standard: C++17 / POSIX API

Operating System: Linux (Kernel ≥ 4.15 / Ubuntu / WSL2)

Build & Test Suite: CMake 3.14+ / CTest

Source Repository: <https://github.com/priyansudas07/Wipro-Embedded-Training>

Submitted in partial fulfillment of the Wipro Embedded Training curriculum requirements.

Date of Submission: September 2026

1. PROJECT OVERVIEW & OBJECTIVES

1.1 Project Description

VDevPulse is a system daemon written in modern C++17 that creates a virtual Linux character device interface (at `/tmp/vdevpulse`) and continuously samples CPU, RAM, and uptime metrics directly from Linux kernel synthetic filesystems (`/proc/stat`, `/proc/meminfo`, `/proc/uptime`). It provides applications with standard POSIX device file operations (`open`, `read`, `write`, `close`) along with virtual IOCTL command handling and thread-safe logging.

1.2 Problem Context & Motivation

Embedded Linux nodes in industrial automation, automotive controllers, and edge gateways require constant monitoring of resource utilization to avoid system brownouts, thrashing, and process starvation. While several monitoring tools exist, they have practical limitations in embedded deployment:

Tool / Approach	Operational Characteristic	Key Limitation in Embedded Systems
top / htop	Interactive terminal user interface	Cannot be programmatically queried as a standard character device stream.
Python / Go daemons	User-space background scripts	Significant memory footprint (50–200 MB RSS), runtime interpreter dependencies.
Prometheus Node Exporter	HTTP/TCP metrics exporter	Requires network stack overhead, socket listeners, and scrape configurations.
Native Kernel Modules (.ko)	In-kernel C character drivers	Requires root compilation against kernel headers; errors cause kernel panics.
VDevPulse (This Project)	Native C++17 POSIX daemon	Zero external dependencies, < 15 MB RSS, standard POSIX device interface.

1.3 Scope of the Project

- In Scope:** Creation of virtual character device node using POSIX FIFO pipes; real-time parsing of `/proc/stat`, `/proc/meminfo`, and `/proc/uptime`; virtual IOCTL command dispatch (`0x8001`, `0x8002`, `0x8003`); dynamic JSON policy parsing; thread-safe multi-level logging; automated CMake CTest suite; systemd service configuration.
- Out of Scope:** Kernel-space driver compilation requiring `insmod`, eBPF in-kernel tracing, and remote network telemetry broadcasting.

2. REQUIREMENTS & DEVELOPMENT PLAN

2.1 Functional Requirements

Req ID	Subsystem	Requirement Description	Priority
FR-01	Device Manager	Create and initialize named FIFO device node at <code>/tmp/vdevpulse</code> using <code>mkfifo()</code> .	High
FR-02	Device Manager	Open device node with <code>O_RDWR O_NONBLOCK</code> flags to prevent blocking on writes.	High
FR-03	Device Manager	Write sampled telemetry payload strings to virtual device node via <code>write()</code> .	High
FR-04	Device Manager	Read user messages from device node via non-blocking <code>read()</code> buffer.	Medium
FR-05	Device Manager	Support virtual IOCTL commands: <code>START (0x8001)</code> , <code>STOP (0x8002)</code> , <code>RESET (0x8003)</code> .	High
FR-06	Device Manager	Maintain device state machine tracking <code>STOPPED</code> , <code>RUNNING</code> , and <code>PAUSED</code> .	Medium
FR-07	Telemetry Monitor	Parse <code>/proc/stat</code> tick line to compute aggregate percentage CPU utilization.	High
FR-08	Telemetry Monitor	Parse <code>/proc/meminfo</code> (MemTotal, MemFree, MemAvailable) to compute RAM in MB and %.	High
FR-09	Telemetry Monitor	Parse <code>/proc/uptime</code> to extract total system uptime in integer seconds.	Medium
FR-10	Telemetry Monitor	Print formatted telemetry dashboard to <code>stdout</code> during active daemon loop.	High
FR-11	Config Parser	Load device path, sampling rate, and memory limit from JSON policy file.	Medium
FR-12	Config Parser	Automatically use compiled fallback defaults if configuration file is absent.	High
FR-13	Logger	Provide 5 logging severity channels: <code>INFO</code> , <code>SUCCESS</code> , <code>WARNING</code> , <code>ERROR</code> , <code>DEVICE</code> .	Medium
FR-14	Logger	Synchronize all logging writes across threads using <code>std::mutex</code> and RAIL lock guard.	High

Req ID	Subsystem	Requirement Description	Priority
FR-15	Main Daemon	Handle <code>SIGINT</code> and <code>SIGTERM</code> signals for clean daemon exit and FIFO node removal.	High

2.2 Non-Functional Requirements

- **Sampling Latency:** Telemetry collection cycle must execute in under 5 milliseconds.
- **Memory Footprint:** Resident Set Size (RSS) must remain below 15 MB.
- **Portability:** Standard C++17 code running on Linux kernels ≥ 4.15 without external libraries.
- **Reliability:** Graceful error handling; no uncaught exceptions; proper RAII file descriptor cleanup.

2.3 Development Timeline & Milestones

Milestone	Focus Area	Key Deliverables	Status
Milestone 1	Concept & Requirements	Problem definition, PRD, functional requirements table, project scope.	Completed
Milestone 2	System Design & Interfaces	Architecture blueprint, UML diagrams, C++ class declarations (<code>include/</code>).	Completed
Milestone 3	Core Implementation	Kernel parsers, Device Manager FIFO I/O, Logger, CLI dispatcher (<code>src/</code>).	Completed
Milestone 4	Testing & Verification	CTest unit test suite (<code>tests/</code>), 100% test pass verification.	Completed
Milestone 5	Deployment & Reporting	systemd service file, 6-stage documentation, presentation guide.	Completed

3. SYSTEM DESIGN & ARCHITECTURE

3.1 Architecture Overview



3.2 Core Data Structures

The system defines the following primary data structures in the header files:

```

// include/vdevpulse/telemetry_monitor.hpp
struct SystemTelemetry {
    double cpu_usage_pct    = 0.0; // Calculated CPU utilization %
    long   memory_total_mb  = 0;    // Total RAM installed (MB)
    long   memory_used_mb   = 0;    // Consumed RAM (MB)
    long   memory_free_mb   = 0;    // Available RAM (MB)
    double memory_usage_pct = 0.0;  // RAM utilization %
    long   uptime_seconds   = 0;    // System uptime (seconds)
};

// include/vdevpulse/config.hpp
struct VDevConfig {
    std::string device_name      = "vdevpulse";
    std::string device_path     = "/tmp/vdevpulse";
    int         sampling_rate_ms = 1000;
    bool        enable_cpu_telemetry = true;
    bool        enable_memory_telemetry = true;
    long        max_memory_threshold_mb = 4096;
};

// include/vdevpulse/device_manager.hpp
enum class DeviceState { STOPPED, RUNNING, PAUSED };

constexpr unsigned long VDEV_IOCTL_START = 0x8001;
  
```

```
constexpr unsigned long VDEV_IOCTL_STOP = 0x8002;  
constexpr unsigned long VDEV_IOCTL_RESET = 0x8003;
```

3.3 Component Responsibilities

- **DeviceManager:** Handles creation, opening, non-blocking reading, writing, and deletion of the FIFO device node, and maintains the internal device operational state.
- **TelemetryMonitor:** Reads raw lines from `/proc/stat`, `/proc/meminfo`, and `/proc/uptime`, parses key fields, computes utilization ratios, and renders the terminal dashboard.
- **Logger:** Singleton log engine ensuring synchronized multi-threaded log writes to both console and `vdevpulse.log`.
- **ConfigParser:** Reads runtime configuration from JSON format, falling back safely to compiled defaults if the file is absent.

4. IMPLEMENTATION DETAILS

4.1 Telemetry Parsing Mechanics

CPU Metric Calculation

The daemon reads the aggregate `cpu` line from `/proc/stat`:

```
std::ifstream proc_stat("/proc/stat");
if (proc_stat.is_open()) {
    std::string cpu_label;
    long user, nice, system, idle, iowait, irq, softirq, steal;
    proc_stat >> cpu_label >> user >> nice >> system >> idle >> iowait >> irq >>
    softirq >> steal;
    long idle_time = idle + iowait;
    long non_idle_time = user + nice + system + irq + softirq + steal;
    long total_time = idle_time + non_idle_time;
    if (total_time > 0) {
        t.cpu_usage_pct = (static_cast<double>(non_idle_time) / total_time) * 100.0;
    }
}
```

RAM Metric Calculation

The daemon reads `/proc/meminfo` and extracts `MemTotal`, `MemFree`, and `MemAvailable`. It uses `MemAvailable` to calculate available memory, as it takes into account reclaimable page caches and inactive memory.

```
long usable_free_kb = (available_kb > 0) ? available_kb : free_kb;
t.memory_total_mb = total_kb / 1024;
t.memory_free_mb = usable_free_kb / 1024;
t.memory_used_mb = t.memory_total_mb - t.memory_free_mb;
t.memory_usage_pct = (static_cast<double>(t.memory_used_mb) / t.memory_total_mb) *
100.0;
```

4.2 Virtual Device Driver & IOCTL Emulation

The virtual character device is created using POSIX `mkfifo()`. To prevent the daemon from blocking when writing without a connected reader, the file descriptor is opened with `O_RDWR | O_NONBLOCK`.

```
bool DeviceManager::openDevice() {
    device_fd_ = open(device_path_.c_str(), O_RDWR | O_NONBLOCK);
    if (device_fd_ < 0) return false;
    state_ = DeviceState::RUNNING;
    return true;
}
```

4.3 Technical Issues Encountered & Resolutions

Issue	Subsystem	Root Cause	Applied Solution
DEV-01	DeviceManager	<code>open()</code> in write-only mode blocked indefinitely without an attached reader.	Used <code>O_RDWR O_NONBLOCK</code> flags on the FIFO node.
DEV-02	ConfigParser	Daemon threw file access errors when optional configuration file was missing.	Added <code>std::filesystem::exists()</code> check with default parameter fallback.
DEV-03	Logger	Buffered log outputs were delayed in terminal output.	Flushed each entry explicitly with <code>std::endl</code> .

5. VERIFICATION & TESTING

5.1 Unit Test Specifications

The project includes an automated test suite executed through CMake's CTest harness. Two standalone test fixtures verify the core subsystems:

Test Executable	Test ID	Test Action & Assertion	Result
device_test	TC-DEV-01	Verify FIFO creation at test path (<code>/tmp/vdevpulse_unittest</code>).	PASS
	TC-DEV-02	Verify device opens and state transitions to <code>RUNNING</code> .	PASS
	TC-DEV-03	Verify writing test string payload succeeds without blocking.	PASS
	TC-DEV-04	Verify <code>sendIoctl(VDEV_IOCTL_START)</code> returns success and retains state.	PASS
telemetry_test	TC-TEL-01	Assert total parsed memory > 0 MB.	PASS
	TC-TEL-02	Assert consumed memory is non-negative and $\leq$ total memory.	PASS
	TC-TEL-03	Assert system uptime value parsed from <code>/proc/uptime</code> ≥ 0 seconds.	PASS

5.2 Test Execution Output

CTest execution log from local build environment:

```
Test project /mnt/f/Projects/Wipro-Embedded-Training/Wipro Individual
Project/vdevpulse/build
  Start 1: device_test
1/2 Test #1: device_test ..... Passed    0.02 sec
  Start 2: telemetry_test
2/2 Test #2: telemetry_test ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 2
Total Test time (real) = 0.03 sec
```

6. DEPLOYMENT, LIMITATIONS & CONCLUSION

6.1 Production systemd Service Configuration

To run VDevPulse as a background service on Linux systems, install the following service definition:

Service File: `/etc/systemd/system/vdevpulse.service`

```
[Unit]
Description=VDevPulse Linux Virtual Device & System Telemetry Daemon
After=network.target local-fs.target

[Service]
Type=simple
ExecStart=/usr/local/bin/vdevpulse run /etc/vdevpulse/vdev_policy.json
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

6.2 Summary of Achievements

- Implemented a fully functioning Linux virtual character device daemon in modern C++17.
- Directly parsed Linux kernel `/proc` interfaces for CPU, RAM, and uptime metrics.
- Emulated POSIX character device operations and IOCTL command dispatch in user-space.
- Implemented thread-safe multi-channel logging using standard C++ synchronization primitives.
- Achieved 100% pass rate on automated CTest unit testing.
- Structured complete 6-stage SDLC engineering documentation and presentation materials.

6.3 System Limitations & Future Work

- **Limitation:** The virtual device node is implemented as a user-space POSIX FIFO rather than an in-kernel character driver (`cdev`).
- **Limitation:** Telemetry collection is currently limited to the local host machine.
- **Future Work:** Port the device manager to a native Linux Kernel Module (`.ko`) using `cdev_add` , `file_operations` , and kernel IOCTL handlers.
- **Future Work:** Add gRPC export capability to broadcast telemetry data to remote monitoring systems.

Conclusion: VDevPulse successfully demonstrates the integration of Linux system programming, POSIX character device interfaces, kernel procs parsing, and modern C++17 architecture in a clean, resource-efficient package suitable for embedded Linux environments.